AgroVision

A Multimodal Agricultural Assistant for Text, Image, and Audio Queries

Mohamed Aymane Bourich-Mouad El Ansari-Youness Anouar

April 2026

Talk Roadmap

- Problem and product vision
- Platform experience and supported modalities
- Model architecture and training pipeline
- Datasets, results, and example output
- Limitations and next steps

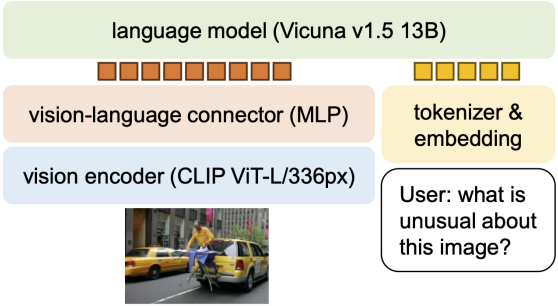
Problem We Are Solving

Core Need

Farmers, students, and agronomy practitioners need quick crop-health guidance without always typing long technical descriptions.

- Disease recognition often depends on both **visual cues** and **domain knowledge**.
- Many users prefer different input modes depending on context:
  - field image + typed question,
  - text-only agronomy question,
  - spoken question when hands are busy.
- The goal is one assistant that can unify these entry points.

Inspiration and Our Adaptation



Inspired by LLaVA-1.5

- CLIP vision encoder
- simple MLP vision-language connector
- efficient visual instruction tuning recipe

Adapted to our setting

- domain goal: agriculture, not general vision chat
- domain data: KisanVaani + PlantVillage
- compute constraints: Qwen2.5-0.5B, LoRA, Colab T4
- design choice: keep all 256 spatial patches for leaf-level detail

Inspired by *Improved Baselines with Visual Instruction Tuning*; figure extracted from papers/2310.03744v2.pdf

Platform Vision

Supported User Flows

- **Image + text**: "What disease is visible and how severe is it?"
- **Text only**: "How much phosphorus per hectare should I apply to wheat?"
- **Audio only**: spoken agronomy question captured by the platform

System Behavior

- Audio is transcribed to text in the platform layer.
- Text and transcribed audio use the same language-reasoning path.
- Image + text uses the full multimodal path.

The repository implements the language and vision-language core; this presentation adds audio as a product-level input channel.

What the Repository Actually Implements

- **Base language model**: `Qwen/Qwen2.5-0.5B-Instruct`
- **Vision encoder**: `openai/clip-vit-large-patch14`
- **Alignment module**: a two-layer **MLP projector**
- **Adaptation**: agriculture LoRA + visual LoRA
- **Training environment**: Google Colab T4 GPU
- **Inference paths**:
  - `generate_text_only(...)`
  - Image-conditioned generation using all visual patches
- **Deployment**: Streamlit frontend + FastAPI inference server

End-to-End System Architecture

Image input	→	CLIP ViT-Large/14 encodes the crop image into patch embeddings
Patch tokens	→	256 spatial tokens are kept; the CLS token is dropped
Projector	→	MLP maps vision embeddings into the LLM hidden space
Language input	→	User text or audio transcript is tokenized with Qwen2.5
Fusion	→	Visual embeddings are concatenated with text embeddings
LLM + LoRA	→	Agriculture LoRA provides domain knowledge; visual LoRA improves multimodal reasoning
Output	→	Diagnosis, severity estimate, management advice, or general agronomy answer

Key design choice

The code keeps **all 256 image patches**, allowing the LLM to attend to lesion patterns, color changes, and texture regions rather than a single pooled visual token.

Training Pipeline in Three Steps

1. **Stage 0 Text-only domain adaptation**
   - Fine-tune a LoRA on agricultural QA text.
   - Dataset used in code: `KisanVaani/agriculture-qa-english-only`.
2. **Stage 1 visual alignment**
   - Freeze ViT and LLM.
   - Train only the projector on image-caption pairs.
3. **Stage 2 multimodal adaptation**
   - Keep ViT frozen.
   - Train projector + new visual LoRA on VQA triplets.

Why this works

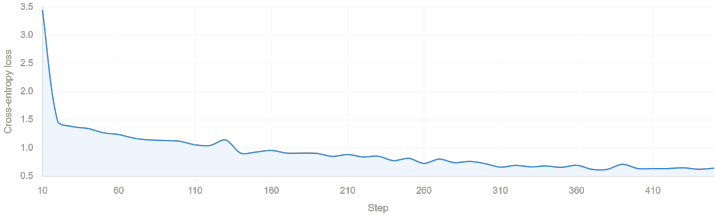
The pipeline first teaches the model agricultural language, then teaches image tokens to "speak" the LLM embedding space, then tunes multimodal reasoning.

Stage 0: Prepare the Agriculture-Aware LLM

Component	Details
Train	LoRA on base LLM
Data	Agriculture text corpus
Loss	Causal LM (next token prediction)
Result	LLM that knows plant diseases, agronomic terminology

- Source in code: `KisanVaani/agriculture-qa-english-only`
- Processed rows in logs: **22,574 usable**; training JSONL subset: **8,000**
- Stage 0 setup: **450 max steps**, effective batch size **16**

Stage 0 Plot: LLM Fine-Tuning



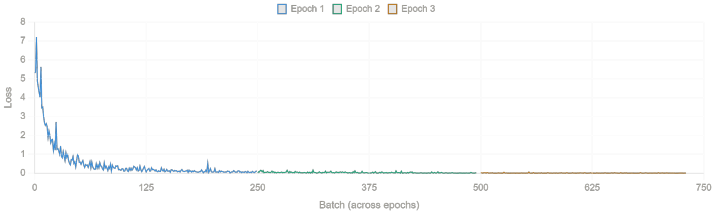
What changed

- LoRA trained on **8,000** agricultural QA samples
- Sharp early drop, then steady convergence
- Final logged train loss: **0.9349**

Stage 1: Train the Visual Projector

Component	Details
Freeze	ViT + LLM
Train	MLP projector only
Data	(image, caption) pairs from disease dataset
Loss	Causal LM on captions given image tokens
Result	Projector that speaks the LLM's embedding language
Purpose	Alignment stage — teach image tokens to look like text tokens

Stage 1 Plot: Caption Alignment



Interpretation

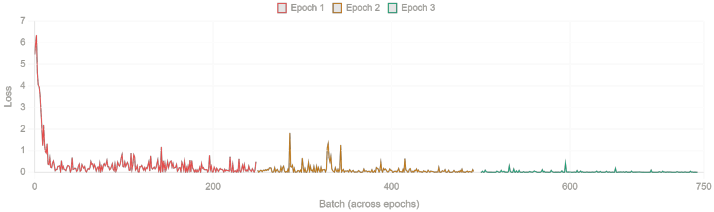
This is the cleanest convergence phase in the pipeline and shows that the projector successfully learns the vision-to-language bridge.

Stage 2: Fine-tune End-to-End with LoRA

Component	Details
Freeze	ViT
Train	MLP projector + new LoRA layers in LLM
Data	(image, question, answer) triplets — disease VQA
Loss	Causal LM on answers
Result	Full multimodal agriculture assistant

- Stage 2 set in logs: **1,000** generated VQA triplets from PlantVillage
- **4** QA pairs sampled per image from 5 templates

Stage 2 Plot: VQA Alignment



Interpretation

The first epoch is noisier because VQA is harder than captioning, but the run still converges cleanly by epoch 3.

Training Snapshot

Phase	Epoch / Run	Metric	Value
Text LoRA	final run	train loss	0.9349
Projector Stage 1	epoch 1	avg loss	0.5500
Projector Stage 1	epoch 2	avg loss	0.0347
Projector Stage 1	epoch 3	avg loss	0.0090
Stage 2 multimodal	epoch 1	avg loss	0.3996
Stage 2 multimodal	epoch 2	avg loss	0.0994
Stage 2 multimodal	epoch 3	avg loss	0.0141

- Report summary: **8.8M trainable parameters**, about **1.75%** of a 502M-parameter model.
- Stage 0 runtime reported: **1007 seconds** with about **7.15 samples/s**.

Demo - Text Only

Demo - Image & Text

Demo - Audio

Demo - PDF File

Why This Design Is Practical

- **Efficient**: LoRA keeps training lightweight relative to full fine-tuning.
- **Modular**: vision encoder remains frozen; only projector and adapters are trained.
- **Accessible**: one platform supports typed, visual, and spoken interaction.
- **Deployable**: repository references Streamlit frontend and API-based serving.
- **Domain-focused**: agricultural QA knowledge is injected before multimodal training.

Current Limitations

- Multimodal supervision is partly template-generated, so linguistic diversity is limited.
- PlantVillage is curated and clean; field images in real farms may be harder.
- Stage 0 stops when the loss is at about **0.9**, so the language adapter has not yet fully covered the full text dataset.

Next Steps

1. Connect live ASR to the deployed frontend for real spoken queries.
2. Increase Stage 0 training beyond 450 steps and add validation-based early stopping.
3. Validate on real field images beyond PlantVillage.
4. Add multilingual prompts and speech support.
5. Replace template-generated captions and VQA pairs with expert-reviewed supervision.
6. Consider a larger backbone such as `Qwen2.5-1.5B` for stronger reasoning.

Takeaway

Final Message

AgroVision is a strong prototype of a **multimodal agricultural assistant**:

- trained first on agricultural text knowledge,
- aligned to **PlantVillage** crop-disease images,
- and presented as a platform that supports **text**, **image + text**, and **audio** interaction.

One assistant, multiple field-friendly ways to ask for help.